

GAS Data Warehouse

Prod → Stage Synchronization Pipeline

Technical Documentation — ADF Pipeline: V2-GAS-Prod-Stage-Pipeline

Data Warehouse: gas_db (Azure Cosmos DB for PostgreSQL – Citus)

1. Purpose & Overview

This pipeline keeps the Stage schema tables in gas_db synchronized with their corresponding Production schema tables. Production is the source of truth; Stage is a working copy used downstream by the Bronze→Silver→Gold medallion pipeline and by ad-hoc analysis. The pipeline is audit-driven: a control table records, per source table, how to extract it, whether to upsert or fully reload it, and pre/post-sync row counts and watermarks so drift can be detected.

At a glance, the pipeline does four things for every table it manages, in this order:

- Snapshots the Prod table's current watermark/row-count state into the audit table (pre-sync).
- Copies data from Prod → Stage, either as a full reload or an incremental upsert, decided per-table.
- Re-snapshots the Prod table's state after the copy (post-sync), so any change during the copy window is visible.
- Falls back to an equivalent Data Flow-based path if the primary Lookup-based path fails.

One dimension table (dim.userlocation) is handled outside the audit-driven loop, as a simple hardcoded truncate-and-reload Copy activity at the end of the pipeline.

2. Control Table: fact.prod_to_stage_migration_audit

This is the brain of the pipeline. Every table it drives has one row here, and the row tells the pipeline exactly what to do — nothing about extraction logic is hardcoded per table inside the pipeline itself.

Column (inferred)	Role in the pipeline
table_schema / table_name	Identifies the Prod table being synced; mirrored 1:1 into Stage.
dependency_level / dependency_count	Drives processing ORDER. The lookup query sorts by these so parent/referenced tables sync before tables that depend on them — this is why the ForEach is sequential, not parallel.
sql_source_query	The actual SELECT run against Prod. Can be a plain “SELECT * FROM schema.table” (full snapshot) or a filtered/incremental query (e.g. watermark-bound).
key_columns	JSON array of key column(s) for the table. Presence/absence is the sole switch between Upsert and Insert logic (see §4).

Column (inferred)	Role in the pipeline
referenced_by	Other tables that depend on this one. Used as a safety check before truncating (see §4).
sql_aggregate	A SQL fragment computing MAX(insert col), MAX(update col), MAX(id), COUNT(*) — injected into the watermark-refresh queries as max_insert / max_update / max_id / record_count.
insert_watermark_value / update_watermark_value / watermark_integer_value / record_count	Pre-sync snapshot of Prod, captured immediately before the copy.
post_sync_insert_watermark_value / post_sync_update_watermark_value / post_sync_watermark_integer_value / post_sync_record_count	Post-sync snapshot of Prod, captured immediately after the copy — compares against the pre-sync values to catch drift.
sysupdatetime / sysupdatetime_after_migration	Timestamps of the pre- and post-sync snapshots.

Companion stored procedure: fact.usp_prod_to_stage_migration_audit()

Called after each watermark refresh. It is not shown in this ARM template (lives in the database), but based on where it's invoked it recalculates/reconciles audit-table metadata after the raw watermark UPDATE — likely dependency_level/dependency_count and any derived flags — before the pipeline reads the row back.

3. Activity-by-Activity Walkthrough

3.1 GAS_Stage_Tables_Lookup

A Lookup activity that selects the list of tables to process this run. The query:

- Deduplicates fact.prod_to_stage_migration_audit with ROW_NUMBER() partitioned by (table_schema, table_name), keeping rn = 1.
- Scoped by exclusion rather than inclusion: a NOT IN list removes 18 tables from the run — mostly non-fact/dim housekeeping tables (stage_watermark, prod_to_stage_migration_audit, prod_to_stage_migration_audit_bkp), reference/lookup tables (device, weather, company, datedim, holidays, menuentities, menuentities_0912), and a few large or separately-handled tables (userbehaviour, deviceevent, userlocation, peripheral, location_statistics, duplicate_items_master, item_modifier_group_modifier_mapping). Every other table in the audit table is in scope by default.
- Two earlier scoping approaches are preserved as commented-out lines directly above the active filter: a sysupdatetime_after_migration IS NULL check (only sync tables not yet migrated) and a narrow inclusion list limited to itemcategory/userbehaviour/deviceevent. Worth keeping these commented rather than deleting — they document how the scope evolved and give a fast way to revert to a smaller test run.
- Orders by dependency_level, then dependency_count, then table_schema/table_name — this ordering is what the downstream sequential ForEach relies on to respect table dependencies.

```
-- Current shape of the lookup query
WITH unique_records AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY table_schema, table_name
```

```

        ORDER BY table_schema, table_name) AS rn
FROM fact.prod_to_stage_migration_audit
WHERE table_name NOT IN (
    'userbehaviour', 'deviceevent', 'userlocation', 'device', 'weather',
    'stage_watermark', 'peripheral', 'prod_to_stage_migration_audit',
    'company', 'datedim', 'menuentities', 'menuentities_0912', 'holidays',
    'prod_to_stage_migration_audit_bkp', 'location_statistics',
    'duplicate_items_master', 'item_modifier_group_modifier_mapping'
)
-- AND sysupdatetime_after_migration IS NULL    -- earlier scoping approach, kept for reference
-- AND table_name IN ('itemcategory', 'userbehaviour', 'deviceevent') -- earlier narrow test scope
)
SELECT * FROM unique_records
WHERE rn = 1
ORDER BY dependency_level, dependency_count, table_schema, table_name;

```

3.2 LoopThroughTables (ForEach, sequential)

Iterates the list from §3.1 one table at a time (isSequential = true). Sequential execution is required here — processing in dependency order only works if one table finishes before the next starts. For each table item, the following sub-activities run.

3.2.1 GetMaxValues (Lookup) — primary pre-sync watermark refresh

Runs a multi-statement batch against Prod for the current table:

- Computes max_insert / max_update / max_id / record_count using item().sql_aggregate against item().table_schema.item().table_name.
- UPDATES fact.prod_to_stage_migration_audit with those values as the pre-sync watermark, and stamps sysupdatetime = now().
- CALLs fact.usp_prod_to_stage_migration_audit() to reconcile audit metadata.
- Re-SELECTs the audit row for this table and returns it as firstRow for downstream activities.

3.2.2 UpsertOrInsert (IfCondition)

Branches purely on whether firstRow.key_columns is null/empty:

Condition	Branch	What happens
key_columns is NULL or empty JSON array	Insert (ifTrue)	Copy activity, writeMethod = CopyCommand (bulk load, no dedup). Prod → Stage using the table's sql_source_query.
key_columns has one or more columns	Upsert (iffalse)	Copy activity, writeMethod = Upsert with upsertSettings.keys = the JSON key_columns array — merges into Stage instead of a raw append.

Both branches share a conditional pre-copy safeguard on the Stage sink:

```

@if(
    and(
        equals(sql_source_query, 'SELECT * FROM ' + table_schema + '.' + table_name),
        empty(json(referenced_by))
    ),
    'TRUNCATE TABLE ' + table_schema + '.' + table_name + ';',

```

```
'SELECT 1 as rn;' -- no-op, table is left as-is
)
```

In plain terms: Stage is only truncated before reload when the source query is an unfiltered full snapshot AND nothing else in the audit table lists this table under referenced_by. If the table is a full snapshot but something else depends on it, the truncate is skipped — this protects against truncating a table mid-loop that a not-yet-processed dependent table still needs to read.

Source dataset: GAS_Prod_Any_PostgreSqlTable (parameterized by table_schema/table_name). Sink dataset: GAS_Stage_Any_PostgreSqlTable (same parameterization) — both are generic, reusable datasets driven entirely by the loop item, not per-table hardcoded datasets.

3.2.3 UpdateMaxValuesAfterMigration (Lookup) — post-sync watermark

Runs after the copy succeeds. Repeats the same MAX/COUNT aggregate against Prod and writes the results into the post_sync_* columns, stamping sysupdatetime_after_migration. Comparing pre-sync vs post-sync values is how drift during the copy window is detected (e.g., new rows inserted into Prod while the copy was running).

3.2.4 Fallback path — runs only if GetMaxValues fails

UpdateAuditTable is wired with dependsOn: GetMaxValues — Failed. It only fires when the primary Lookup-based watermark refresh in §3.2.1 errors out. It re-implements the same pre-sync watermark logic as a Data Flow (UpdateGASAuditTable, 16 cores, General compute) instead of a Lookup, passing table_schema / table_name / sql_aggregate as parameters.

From there the pipeline effectively repeats the same sequence on the fallback rail:

Primary rail	Fallback rail (only on primary failure)
GetMaxValues (Lookup)	UpdateAuditTable (Data Flow)
UpsertOrInsert → Insert / Upsert	GetMaxValues2 (Lookup, re-calls usp_prod_to_stage_migration_audit)
UpdateMaxValuesAfterMigration (Lookup)	UpsertOrInsert2 → Insert2 / Upsert2 (identical branch logic)
—	PostSyncUpdateAuditTable (Data Flow, post-sync watermark)

Practical read: a Lookup activity has a result-size/behavior ceiling that a heavier aggregate query can hit for large tables (e.g. deviceevent at hundreds of millions of rows). The Data Flow fallback gives the same aggregate a Spark compute context to run in instead, at the cost of spinning up a data flow cluster. If a table only ever needs the fallback rail, that's worth flagging — it may indicate the primary Lookup path is undersized for that table's aggregate cost.

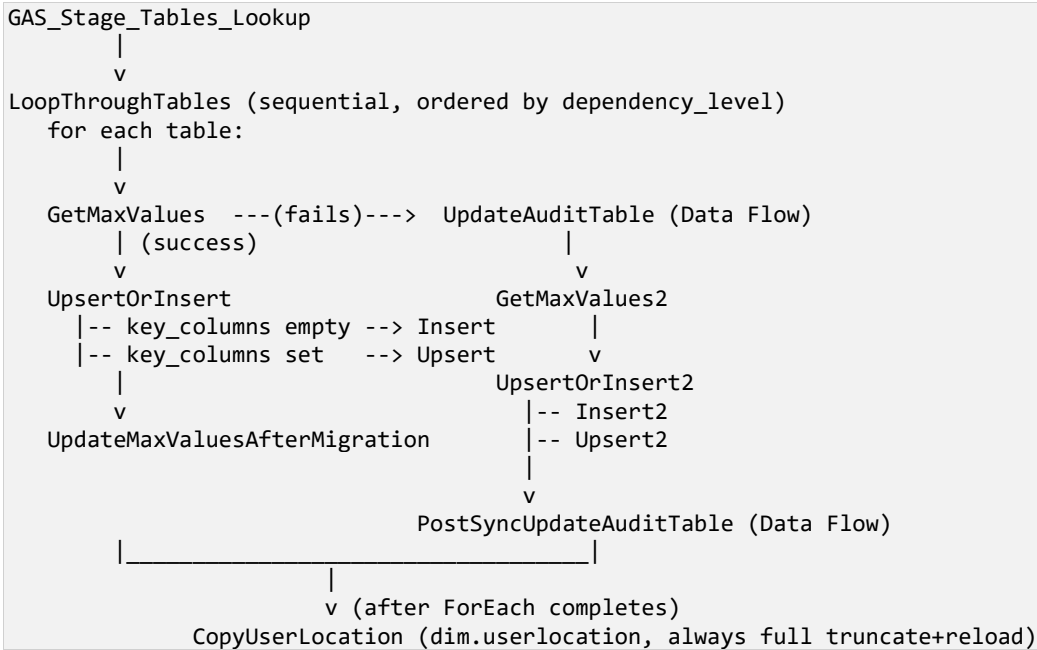
3.3 CopyUserLocation (Copy)

Runs once, after the entire LoopThroughTables ForEach completes. Unlike every other table, this is not audit-table-driven: it's a hardcoded full Copy of dim.userlocation.

- Source: GAS_Prod_Dim_PostgreSqlTable parameterized to userlocation (plain SELECT, no aggregate/watermark logic).

- Sink: GAS_Stage_Dim_PostgreSQLTable, preCopyScript unconditionally truncates dim.userlocation, writeMethod = CopyCommand.
- No Upsert option, no pre/post watermark tracking, no dependency-order participation — it is always a full truncate-and-reload, and it always runs last.

4. End-to-End Flow Diagram (textual)



5. Design Notes & Things Worth Confirming

- Sequential ForEach + dependency_level ordering is the mechanism that keeps referential integrity intact across the loop — do not change isSequential to false without re-checking the truncate safeguard in §3.2.2.
- Insert vs. Upsert is entirely data-driven via key_columns in the audit table — to change a table's sync behavior, update that row, not the pipeline.
- The truncate-before-reload guard only checks referenced_by and an exact full-snapshot query match; a table with a filtered sql_source_query that still needs a truncate (e.g., a periodic full refresh with a WHERE clause) will silently skip the truncate and append/duplicate instead — worth keeping in mind when writing new sql_source_query values.
- The Lookup vs. Data Flow duplication (GetMaxValues/GetMaxValues2, UpsertOrInsert/UpsertOrInsert2) means any future logic change has to be made in two places to stay consistent.
- GAS_Stage_Tables_Lookup now scopes by exclusion (NOT IN 18 named tables) rather than by inclusion, meaning any new table added to fact.prod_to_stage_migration_audit is in scope automatically unless explicitly excluded — opposite failure mode from the old inclusion-list approach: instead of silently leaving new tables out, it will silently pick new tables up. Worth a periodic check that nothing lands in scope unintentionally.
- dim.userlocation is intentionally outside the audit framework — if it ever needs incremental sync or dependency-aware ordering, it will need to be migrated into fact.prod_to_stage_migration_audit like the other tables.

6. Reference: Datasets Used

Dataset	Points at	Used by
GAS_Stage_factMigrationAudit	fact.prod_to_stage_migration_audit (Stage)	GAS_Stage_Tables_Lookup, GetMaxValues, UpdateMaxValuesAfterMigration, GetMaxValues2
GAS_Prod_Any_PostgreSqlTable	Parameterized Prod schema.table	Insert/Upsert/Insert2/Upsert2 (source)
GAS_Stage_Any_PostgreSqlTable	Parameterized Stage schema.table	Insert/Upsert/Insert2/Upsert2 (sink)
GAS_Prod_Dim_PostgreSqlTable	Prod dim table, parameterized to userlocation	CopyUserLocation (source)
GAS_Stage_Dim_PostgreSqlTable	Stage dim table, parameterized to userlocation	CopyUserLocation (sink)